RTL-to-GDS on GF180MCU

LibreLane + Docker in practice

Mauricio Montanares

April 22, 2026

Scientist, IHP

What you will learn

- What RTL-to-GDS means end-to-end, and where LibreLane sits.
- How to get the full toolchain in **one Docker command**.
- How to configure LibreLane for a GF180MCU full-chip design (the config.yaml anatomy).
- How to run the flow and read DRC / LVS / timing signoff.

Prerequisites: Linux host, Docker installed, ∼40 GB free disk. *No prior EDA knowledge assumed.*

Agenda

The RTL-to-GDS pipeline

The stack, the container, the PDK

LibreLane config – the heart of the flow

Running the flow

Flow stages up close

Results, pitfalls, and next steps

The RTL-to-GDS pipeline

RTL and GDS – two ends of the same pipeline

RTL

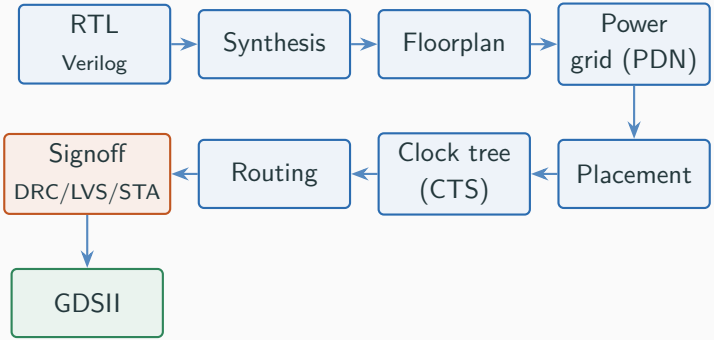
A text description of a circuit in Verilog / SystemVerilog.
Says *what* the circuit does.

GDS / GDSII

A binary layout file – every polygon on every layer.
What the foundry turns into masks.

RTL-to-GDS = automating the path from one to the other.

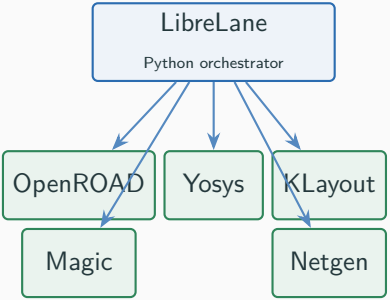
The pipeline, at a glance



LibreLane is the conductor, not the orchestra

LibreLane is a *Python program* that does **no EDA work itself**. It reads one configuration file and, for each stage, calls the open-source tool that does the real work:

- **Yosys** – RTL → gate-level netlist.
- **OpenROAD** – floorplan, PDN, placement, CTS, routing, STA, IR drop.
- **Magic** – GDS streamout and DRC.
- **KLayout** – DRC cross-check, antenna, XOR, renders.
- **Netgen** – LVS (layout vs schematic).



A conductor does not play instruments. LibreLane does not place cells.

What you get at the end

A successful flow run hands you:

- `final/gds/chip_top.gds` – the layout.
- `final/def/` – routed netlist + placement.
- `final/lib/` – timing libraries (extracted).
- `final/spef/` – parasitics for post-layout STA.
- `final/sdf/` – back-annotation for gate-level sim.
- `metrics.csv` – one row per metric, for CI diffs.

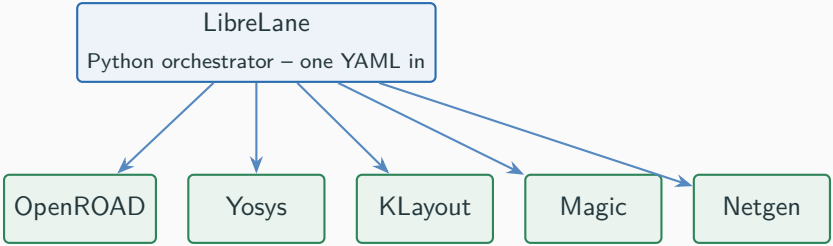
Screenshot placeholder

`slides/figures/final_layout_render.png`

Final GDS rendered by KLayout.

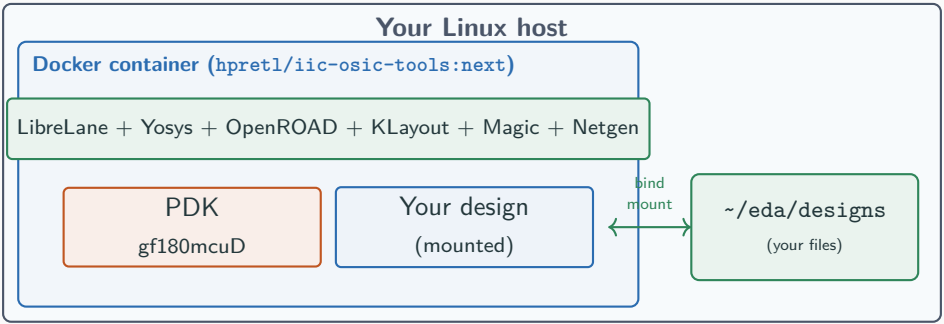
The stack, the container, the PDK

The full stack in one picture



The CLI is literally `librelane config.yaml`. Everything below the top box is what does the *actual* EDA work; LibreLane reads the config, sequences the tools, passes files between them, and writes one `metrics.csv`.

Host, container, and your files



The container is the **environment**; the bind-mount is the **bridge** – edits on the host appear instantly inside.

Container bootstrap – four commands

```
# 1. One-time image pull (~15 GB)
docker pull hpretl/iic-osic-tools:next

# 2. Start a long-running, headless container
docker run -d --name gf180 \
    -v ~/eda/designs:/foss/designs:rw \
    --user $(id -u):$(id -g) \
    hpretl/iic-osic-tools:next --skip sleep infinity

# 3. (Optional) interactive shell for debugging
docker exec -it gf180 bash

# 4. Non-interactive use (what the rest of the deck does)
docker exec gf180 bash -lc '<command>'
```

GF180MCU: two PDK gotchas to know upfront

- The image ships `gf180mcuD` (5 V MCU flavour) via **ciel**. Good to go.
- The full-chip template needs two extra I/O cells (`gf180mcu_ws_io__dvdd`, `__dvss`) that live in the **wafer-space fork**:

```
git clone --depth 1 --branch 1.8.0 \
  https://github.com/wafer-space/gf180mcu.git gf180mcu
```

- The container defaults to IHP SG13G2. Every new shell must re-activate GF180:

```
source sak-pdk-script.sh gf180mcuD gf180mcu_fd_sc_mcu7t5v0
```

LibreLane config – the heart of the flow

The project template: files you actually touch

The YAML below is LibreLane's input. LibreLane parses it and hands the relevant pieces to Yosys, OpenROAD, Magic, KLayout, and Netgen.

Base: `github.com/wafer-space/gf180mcu-project-template`.

Five files matter:

File	Purpose
`src/chip_top.sv`	Top module: padring + core.
`src/chip_core.sv`	Your logic. *Replace this for a custom chip.*
`librelane/config.yaml`	Design-wide settings (RTL, clock, PDN, macros).
`librelane/slots/slot_1x1.yaml`	Floorplan (die, core, pad order).
`librelane/pdn_cfg.tcl`	PDN script (trust it for now).

Plus a `Makefile` that wraps `librelane` in `make librelane`.

LibreLane v3 YAML – two levels, one command

- `config.yaml` – **design-level** (clock, RTL files, PDN, macros). Rarely changes.
- `slots/slot_*.yaml` – **floorplan-level** (die size, pad order). Changes per shuttle slot.

LibreLane merges both at the command line:

```
librelane librelane/slots/slot_1x1.yaml librelane/config.yaml \
         --pdk gf180mcuD --pdk-root <path> --manual-pdk
```

config.yaml — the header

```yaml
meta:
  version: 3
  flow: Chip

DESIGN_NAME: chip_top
VERILOG_FILES:
  - dir::../src/chip_top.sv
  - dir::../src/chip_core.sv

VDD_NETS: [VDD]
GND_NETS: [VSS]

CLOCK_PORT: clk_PAD
CLOCK_NET:  clk_pad/Y
CLOCK_PERIOD: 40           # ns -> 25 MHz
```

config.yaml – the power grid

```yaml
PDN_VWIDTH: 5
PDN_HWIDTH: 5
PDN_VSPACING: 1
PDN_HSPACING: 1
PDN_VPITCH: 75
PDN_HPITCH: 75

PDN_CORE_RING: true
PDN_CORE_RING_VWIDTH: 25
PDN_CORE_RING_HWIDTH: 25
PDN_CORE_RING_CONNECT_TO_PADS: true

PDN_CORE_VERTICAL_LAYER:   Metal2
PDN_CORE_HORIZONTAL_LAYER: Metal3

PDN_CFG: dir::pdn_cfg.tcl
```

config.yaml – declaring macros

```yaml
MACROS:
  gf180mcu_fd_ip_sram__sram512x8m8wm1:
    gds:  [pdk_dir::libs.ref/gf180mcu_fd_ip_sram/gds/..gds]
    lef:  [pdk_dir::libs.ref/gf180mcu_fd_ip_sram/lef/..lef]
    vh:   [pdk_dir::libs.ref/.../sram__blackbox.v]
    lib:
      "*_tt_025C_5v00": [pdk_dir::.../sram__tt_025C_5v00.lib]
      "*_ss_125C_4v50": [pdk_dir::.../sram__ss_125C_4v50.lib]
      "*_ff_n40C_5v50": [pdk_dir::.../sram__ff_n40C_5v50.lib]
    instances:
      i_chip_core.sram_0: {location: [490, 500], orientation: N}
      i_chip_core.sram_1: {location: [970, 500], orientation: E}
```

Slots: a wafer-space convention, not a LibreLane one

A **slot** is a pre-reserved rectangle on wafer-space's MPW shuttle. Three are standard: `1x1` ($\sim$3.9$\times$5.1 mm), `1x0p5`, `0p5x0p5`. Each `slot_*.yaml` sets `DIE_AREA`, `CORE_AREA`, and the pad lists; everything else stays in `config.yaml`.

Not taping out with wafer-space? Three paths:

1. **Other shuttle** (IHP MPW, TinyTapeout, ...) – use that programme's template; its own die sizes replace slots.
2. **Custom tape-out with padring** – drop `slots/`. Move `FP_SIZING`, `DIE_AREA`, `CORE_AREA` and the `PAD_*` lists into `config.yaml`. Pick any die size that fits your logic.
3. **Bare macro (no padring)** – switch to `meta.flow: Classic`. No pads, no slots, no sealring, no wafer-space PDK fork. That is what the counter notebook does.

slot_1x1.yaml – the floorplan

```yaml
FP_SIZING:  absolute
DIE_AREA:   [0, 0, 3932, 5122]        # um
CORE_AREA:  [442, 442, 3490, 4680]    # um

PAD_SOUTH: [clk_pad, rst_n_pad,
            "bidir\\[0\\].pad", ..., "dvss_pads\\[0\\].pad", ...]
PAD_EAST:  [...]
PAD_NORTH: [...]
PAD_WEST:  [...]
```

- `DIE_AREA` = full die outline (includes the sealring).
- `CORE_AREA` = rectangle for standard cells, inside the padring.
- Pad lists read *clockwise* starting from the south-west corner; backslashes escape the Verilog brackets (regex).

Running the flow

The golden path – copy, paste, done

```
docker exec gf180 bash -lc '
  cd /foss/designs/template

  # One-time: clone the wafer-space PDK fork
  make clone-pdk

  # Every shell: activate GF180MCU
  source sak-pdk-script.sh gf180mcuD gf180mcu_fd_sc_mcu7t5v0

  # Run the full RTL-to-GDS flow on slot 1x1
  make librelane \
       SLOT=1x1 \
       PDK=gf180mcuD \
       PDK_ROOT=/foss/designs/template/gf180mcu
'
```

Expected runtime: ∼35–45 min on a modern laptop.

What scrolls by while it runs

- LibreLane prints a banner and numbered steps (`Step 1: Yosys.Synthesis`, ...).
- Yosys: inferred cell counts + area.
- OpenROAD: floorplan, PDN, placement, CTS, routing, STA.
- Magic + KLayout: DRC (the slow parts).
- Netgen: LVS.
- Final banner: run directory + pointer to `metrics.csv`.

Screenshot placeholder

`slides/figures/librelane_run_output.png`

Tail of a successful `make librelane` run.

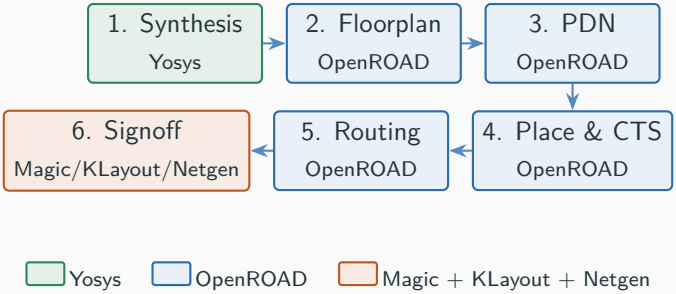
Where artifacts land

```
template/
  librelane/runs/RUN_<timestamp>/   # per-run: logs, DEFs, reports
  final/                            # merged: GDS, LEF, SDC, SDF, SPEF,
                                    # metrics.csv, render/chip_top.png
```

Useful after a successful run:

- `final/gds/chip_top.gds` – the deliverable.
- `final/metrics.csv` – one row per metric.
- `final/render/chip_top.png` – auto-render for posters.

79 steps grouped into six phases



Each phase writes a milestone file; any failure halts the flow. LibreLane is the driver for all six.

Flow stages up close

Phase 1 – Synthesis (Yosys)

LibreLane driving: **Yosys**

- RTL → gate-level netlist using the GF180 standard-cell library (`gf180mcu_fd_sc_mcu7t5v0`).
- Emits instance counts, estimated area, inferred FF / LUT shapes.
- Produces the first mapped `chip_top.v` that feeds every downstream step.

Reference run numbers:

```
design__instance__count,251615
design__instance__count__stdcell,84417
```

Phase 2 – Floorplan & padring

LibreLane driving: **OpenROAD**

- `OpenROAD.Floorplan` reads `DIE_AREA` / `CORE_AREA` and creates an empty chip frame.
- `OpenROAD.Padring` places I/O pads per `PAD_SOUTH/...` order.
- `ODB.ManualMacroPlacement` drops the SRAMs at the coordinates from `config.yaml`.
- `CutRows` + `TapcellInsertion` prepare standard-cell rows.

Screenshot placeholder

`slides/figures/floorplan.png`

OpenROAD: post-floorplan view (padring + SRAM placeholders).

Phase 3 – Power Distribution Network

LibreLane driving: **OpenROAD**

- OpenROAD runs `pdn_cfg.tcl` to draw the grid.
- **Followpin** wires: thin VDD/VSS along every std-cell row.
- **Straps**: Metal2 vertical + Metal3 horizontal across the core.
- **Core ring**: loop around the core that connects to pads.
- Per-macro grids stitch SRAM power pins to the main grid.
- Checker target: `design__power_grid_violation__count = 0`.

Screenshot placeholder

`slides/figures/pdn.png`

Core PDN with straps and ring visible.

Phase 4 – Placement & CTS

LibreLane driving: **OpenROAD**

- **Global placement**: approximate positions (RePlAce).
- **Detailed placement**: legalise onto std-cell rows.
- **Clock Tree Synthesis**: buffer tree from `clk_pad/Y` to every flop, minimising skew.
- **Post-CTS resizing**: fixes hold violations introduced by CTS.

Reference run:

```
design__instance__count__class:clock_buffer,30
clock__skew__worst_setup,0.319   # ns
clock__skew__worst_hold, -0.328  # ns
```

Phase 5 – Routing

LibreLane driving: **OpenROAD**

- **Global routing**: assigns nets to routing regions.
- **Detailed routing**: puts wires on physical tracks.
- **Antenna repair**: inserts diodes on long metals that would damage gates during plasma etching.
- Iterates until zero DRC.

```
route__drc_errors__iter:0, 75
route__drc_errors__iter:5,  0      # clean after 6 iterations
route__wirelength__estimated, 153555  # um
antenna__violating__nets, 0
```

Phase 6 – Signoff (geometry + electrical)

LibreLane driving: **Magic + KLayout + Netgen + OpenSTA**

Geometry (the slow part):

- **Magic DRC** + **KLayout DRC** cross-check.
- **KLayout XOR** – Magic GDS vs KLayout GDS (*0 diff*).
- **Antenna / Density / Sealring** checks.

Electrical:

- **OpenRCX** extracts parasitics (3 corners).
- **OpenSTA** runs timing across 9 PVT + extraction corners.
- **PDNSim** does IR drop.
- **Netgen LVS** – extracted netlist vs synthesized Verilog.
- Zero DRC / LVS / setup / hold / antenna violations = green chip.

Results, pitfalls, and next steps

Headline numbers – reference run

Metric	Value
Die area	3.93×5.12 mm
Core area	3.05×4.24 mm
Core utilisation	10.45 %
Standard-cell instances	84 417
Total instances (incl. padring, taps)	251 615
Macros	4 (2× SRAM + ID + logo)
I/O pads	58
Routed wirelength	153.5 mm
Total power (tt_025C_5v00)	34.3 mW
Clock period / frequency	40 ns / 25 MHz

Source: `final/metrics.csv`, run 2026-04-13.

Signoff checklist – the things that must be zero

Check	Violations
Magic DRC	0
KLayout DRC	0
KLayout XOR (Magic vs KLayout)	0
KLayout antenna / density	0
Magic illegal overlap	0
Netgen LVS	clean
Setup / Hold (all 9 STA corners)	0 / 0
Routing DRC (final iteration)	0
IR drop	0
Max slew (ss_125C corner only)	3 (warning)
Max cap (all corners)	100–103 (warning)

Chip is manufacturable; the warnings are slow-corner fine-tuning.

Reading `metrics.csv` in one grep

```
grep -E '^(magic__drc|klayout__drc|lvs|timing__(setup|hold)_vio__count$|design__(violations|
    die__area|instance__count)|power__total)' \
    ~/eda/designs/template/final/metrics.csv
```

`metrics.json` has the same data in a friendlier shape for scripts. Commit a trimmed copy as a regression baseline.

Four pitfalls you *will* hit

1. **Wrong PDK active.** Error mentions `sg13g2_stdcell`. **Fix:** `source sak-pdk-script.sh gf180mcuD gf180mcu_fd_sc_mcu7t5v0` before `make`.
2. **Missing SRAM / I/O files.** Error says path `'/foss/pdks/ihp-sg13g2/libs.ref/gf180mcu_fd_ip_sram/...'` does not exist. **Fix:** pass `PDK_ROOT=/foss/designs/template/gf180mcu` explicitly.
3. **Unknown I/O cell `gf180mcu_ws_io__...`.** **Fix:** clone the wafer-space PDK fork (`make clone-pdk`).
4. **Flow is absurdly slow.** Magic DRC $\sim$32 min + KLayout antenna $\sim$7 min dominate runtime. **Fix for iteration:** `make librelane-nodrc`; never hand in a chip without one full DRC run.

What to try next

1. **Swap** `chip_core.sv` for your own logic (same ports). Re-run. You now have a custom chip.
2. **Change** `CLOCK_PERIOD` and watch setup/hold slack move.
3. **Swap the slot** to `1x0p5` or `0p5x0p5`, adjust pad lists, re-run – see the floorplan scale.
4. **Submit to a shuttle**: Tiny Tapeout, wafer-space MPW. The GDS you produced is fab-ready.

Further reading: LibreLane docs `https://librelane.readthedocs.io/en/stable/` · Image `https://github.com/iic-jku/IIC-OSIC-TOOLS` · Template `https://github.com/wafer-space/gf180mcu-project-template` · Wafer-space PDK fork `https://github.com/wafer-space/gf180mcu` · GF180 PDK `https://github.com/google/gf180mcu-pdk`.